ENGINEERING PROPOSAL / 2026

MOD/REF PLAN

JobTracker: modularity, structure and internal references

Prepared 11 October 2026. Repository-specific proposal, based on source inspection during the phone UI update. The architecture changes below are proposed work; they have not been applied by this delivery.

Recommendation: improve navigation and reference checks first, then extract one feature at a time behind the existing public interfaces. Keep the working desktop/server business-logic boundary and the plain JavaScript phone client.

The main efficiency gain comes from making ownership explicit: a task should point directly to its entry point, authoritative rule, write operation, persistence owner and meaningful tests. A directory tree alone cannot provide that.

What success looks like

- Find the owner and verification command for a feature from one index.
- Change a business rule in one authoritative place.
- Keep queued edits, stable identities and timer recovery intact during refactoring.
- Detect stale references and forbidden imports automatically.
- Make large files smaller by responsibility, with clear public interfaces.

Reading guide

Pages	Topic
2-3	Repository findings and target architecture
4-5	Reference system and documentation workflow
6-9	Concrete module splits and redundancy removal
10	Testing and development tooling
11-13	Phased roadmap and 26-item backlog
14-15	Migration example, acceptance checks and source index

This plan preserves a personal, self-hosted app. It does not require a web framework rewrite, a new database, a public service, a plugin system or a monorepo toolchain.

01 / Findings grounded in this repository

Useful foundations to preserve

`jobtracker/core` and `jobtracker/services` have no PySide6, UI or server imports in the inspected Python AST. Desktop and server already share `TrackerService`. `timeutils.py`, `sync_policy.py`, the ordered outboxes and shared session-list UI are valuable existing seams.

The recent weight implementation also demonstrates a workable pattern: separate service, pure plot mathematics, a dedicated phone module and cross-language fixtures. Extend these patterns before introducing new abstraction layers.

Highest-value extraction candidates

Current owner	Audit size	Why it costs effort
`web/app.js`	1,577 lines	Navigation, refresh, feature rendering, forms and graph drawing are coupled.
`core/database.py`	1,472 lines	Schema setup, entity persistence, identity mapping and backup merge share one class.
`services/tracker_service.py`	1,148 lines	Timer state, subjects, goals, recurrence, analytics and export share one facade.
`ui/app.py`	803 lines	Window setup, lifecycle, shortcuts, sync coordination and feature wiring.
`server/ops.py`	697 lines	Operation registry, validation, identity lookup, business dispatch and receipts.
`tests/test_ui_smoke.py`	2,522 lines	Broad UI coverage has a large search surface; grouping by feature would improve routing.

Sizes are the inspection snapshot before this turn's final phone edits, not generated complexity scores. File length identifies review candidates; it does not prove a file is defective. The same snapshot contained 43 top-level pytest modules.

Navigation debt

The audit found stale documentation about agenda day semantics, removed heatmaps and test setup. Those details are corrected in this delivery. Their presence shows why references and selected documentation facts need checks, rather than another long document copied across tools.

First move: add a maintained feature map and a small navigation document. Do not begin by moving all files or renaming legacy SQL tables.

02 / Target structure and dependency direction

Organize by responsibility within the existing layers. Features own their rules; adapters own transport and presentation. Keep compatibility facades while modules move underneath them.

```
jobtracker/
  core/                  pure types, time, validation, plot math
    persistence/         connection, schema, entity repositories
  services/              use cases: timer, subjects, goals, analytics, weight
    tracker_service.py   stable composition facade
  sync/                  ordered queue, transport, mirror, orchestration
  ui/                    native window shell, feature panels, shared widgets
server/
  app.py                 composition, middleware, final static mount
  routes/                authenticated reads grouped by feature
  operations/            explicit handler registry grouped by feature
web/
  app.js                 startup, navigation, feature coordination
  features/              subjects, goals, sessions, graphs, weight
  shared/                DOM, formatting, fullscreen, chart primitives
  data/                  API, outbox, mirror, display projection
  styles/                tokens, shell, controls, feature styles
docs/
  index.md               human and agent entry point
  architecture/          boundaries, feature map, data/time/write contracts
  runbooks/              test, build, install, deploy, incident steps
  generated/             checked reference index and dependency summary
scripts/                 repository checks and safe development commands
```

Allowed dependency direction

Presentation and HTTP adapters call service interfaces. Services call pure core utilities and persistence interfaces. Persistence owns SQL and transaction primitives. Core never imports Qt, FastAPI, server code or UI modules. Sync adapts service writes and coordinates mirrors; it does not become the owner of business rules.

A route/controller may compose a use case, but it must not add another rule for goal completion, timer ownership or daily weight identity. JavaScript offline projections remain provisional display logic and are verified against server contract fixtures.

Public surface

Keep `Database`, `TrackerService` and `SyncedTrackerService` import paths stable during extraction. New internal modules should export only named, intentional interfaces. Use `__init__.py` for a small public surface, never wildcard exports. Keep explicit JavaScript imports; avoid a barrel that imports every feature during startup.

Proposed structure: create a folder when it has a clear responsibility and more than one coherent module. Do not create one-file packages solely to make the tree look modular.

03 / A reference system that reduces searching

Better referencing helps when it gives a reliable starting point. It cannot make an agent know runtime behavior automatically. The index should identify the small set of files and contracts to inspect, then let tests and source verify the change.

One feature manifest

Use `docs/architecture/features.json` as the maintained navigation source. JSON uses the Python standard library and does not require a YAML dependency. Give each feature a stable ID; use file paths plus qualified symbols instead of fragile line numbers.

```
{
  "schema_version": 1,
  "features": [{
    "id": "weight.daily-entry",
    "entrypoints": [
      {"path": "web/weight.js", "symbol": "renderWeight"},
      {"path": "jobtracker/ui/widgets/weight_view.py",
       "symbol": "WeightView"}
    ],
    "rule_owner": {
      "path": "jobtracker/services/weight_service.py",
      "symbol": "WeightService.upsert"
    },
    "operations": ["upsert_weight", "delete_weight"],
    "tables": ["weight_measurements", "weight_uid_aliases"],
    "contracts": ["weight.calendar-day", "sync.ordered-outbox"],
    "tests": ["tests/test_weight_daily_edges.py",
              "tests/mobile/weight-picker.test.mjs"]
  }]
}
```

The example is illustrative; verify every actual symbol when the manifest is implemented. Store paths relative to the repository. Include shared dependencies, targeted test commands, read endpoints and related features where they change task scope.

Generate and verify

`check_references.py` should validate paths, Python AST symbols, supported JavaScript exports, unique feature IDs, contract IDs and test paths. Generate a compact Markdown table from the manifest. Link generated rows back to the owner and its tests.

Do not claim to generate a complete JavaScript call graph with a regex. Validate supported export declarations initially; add an AST parser only if the simpler check cannot cover real modules.

Proposed task command: `python scripts/dev.py context weight.daily-entry` prints owners, contracts, focused tests and related features. It should output a short task packet, not concatenate the entire repository. This command is a future deliverable, not a command available today.

04 / Documentation and agent navigation

Separate the kinds of information

Document	Responsibility
Root `AGENTS.md`	Short agent entry point: scope, required checks, safety invariants and links.
`README.md`	Product behavior, setup and everyday commands.
`docs/index.md`	Where to start for each engineering task.
`architecture/features.json`	Machine-readable owners, interfaces, routes, operations and tests.
`architecture/time-model.md`	Logical tracking days, calendar weight days and accepted local-time behavior.
`architecture/write-path.md`	Atomic writes, outboxes, replay receipts, ordering and mirror repair.
`runbooks/*.md`	Reproducible testing, build/install, deployment and recovery steps.
`architecture/decisions/*.md`	Short explanations of durable tradeoffs and rejected alternatives.

Paths in the last six rows are under `docs/`. Keep `CLAUDE.md` as a short pointer to the shared rules if the tool expects that filename. Do not maintain two full sets of engineering instructions.

Use a shared vocabulary

Document the compatibility translation once: Subjects are the legacy `tasks` table; Goals are `todo_tasks` / `TodoTask`; Weight uses `weight_measurements` and the wire name `weights`. Add the table-to-wire map generated from `sync_policy`, rather than manually restating it in many files.

A reference should explain why a reader needs another module: "Writes must use this adapter because it stores the mutation and outbox atomically" is more helpful than "See sync code". Add those comments at important seams, not beside every import.

Keep references current in the same change

When a feature moves, update its manifest row, its public interface documentation and its targeted test map in the same change. Generated output must be reproducible. Check documentation examples against actual method names and registered routes where practical.

Avoid fixed test totals, historical byte sizes and deployment identities in policy documents. Store measured reports separately, with date and measurement scope. Exclude live data, token contents, server addresses and private environment values from generated references.

Done condition: a new contributor or agent can start from `AGENTS.md`, follow one feature row and identify the authoritative write path and test command without a whole-repo search.

05 / Python services and persistence

Extract business responsibilities behind facades

Current responsibility	Proposed owner	Preserve during extraction
Start, stop, switch, heartbeat, recovery	`services/timer_service.py`	Session UID targeting, device ownership and retained unfinished sessions.
Subject CRUD, archive and ordering	`services/subject_service.py`	Existing API and archive/delete distinction.
Goals and milestones	`services/goal_service.py`	Completion gate and reopening rules.
Recurring generation	`services/recurrence_service.py`	Additive, period-idempotent generation.
Bar/agenda aggregation and ranges	`services/analytics_service.py`	Logical-day rules and existing grouping behavior.
Weight history	Existing `weight_service.py`	Calendar-day upsert, aliases, imported precision.
Backup validation and merge	Backup service + persistence importer	Stable identities and atomic restore/queue behavior.

`TrackerService` composes these owners and delegates, so callers keep working. Extract low-state subjects/goals first; handle the timer state machine after those seams prove stable. Avoid leaving two mutable timer states in the facade and extracted service.

Divide persistence by purpose

Start with connection/transaction ownership, idempotent schema setup and backup serialization. Then move entity SQL into repositories sharing one injected connection. Keep `Database` as the public compatibility facade until callers have migrated deliberately. Later, let entry points own database creation instead of relying on the module singleton; preserve early test isolation while changing that seam.

`Database.atomic()` remains the transaction owner. An entity repository must not introduce a second connection or commit independently inside a larger mutation. Preserve UID assignment triggers, feed trigger timing and nested transaction behavior.

Numbered migrations can become a later improvement, after current additive migrations have characterization tests. Do not mix structural extraction with a table rename or schema redesign. Keep `tasks` and `todo_tasks` SQL names for compatibility and document the mapping.

A focused first extraction

Move goal completion and milestone reopening together. Run goal/focus/template tests and operation/sync coverage; then run the required full suite. The facade's outputs, mutation order and wire operations must remain identical. Only after that, remove the copied implementation from the old owner.

Avoid: a generic repository base class for every table, global service locators, or decorators that infer write contracts solely from method names.

06 / Server, sync and explicit contracts

Server composition

Split read endpoints into `server/routes/{context,sessions,graphs,weight,sync}.py`. Keep app creation, authentication middleware, compression, lifespan and static mounting in the composition module. Register API routers before the final `/` static mount so assets cannot shadow data routes.

Split `server/ops.py` into explicit operation modules by domain. Keep replay receipt creation, principal binding, parameter fingerprinting, ordered batch handling and transaction scope central. Each handler returns the current response contract; it does not own replay mechanics.

Maintain one explicit operation registry. Generate operation documentation from it and compare the registry with desktop queue adapters and mobile projection support. Some operations intentionally have no immediate projection; record that explicitly instead of treating every omission as equivalent.

Preserve the queue's error semantics

Result	Meaning	Queue consequence
Applied / replayed	Durable canonical result or same prior receipt	Remove only acknowledged entries.
Recorded skip	Edit/delete targets an already removed row	Acknowledge safely; allow later writes.
State refusal	Business state prevents the requested action	Preserve current documented payload behavior.
Invalid request	Malformed input or missing parent for a create	Stop the batch and expose the precise failure.
Transport / temporary failure	No trustworthy acknowledgement	Retain operations with original IDs and order.

Introduce named internal result types as handlers are extracted. Keep existing HTTP and JSON shapes until a separately tested compatibility change is justified.

Sync modules

Keep transport, queue persistence and orchestration separate. Decompose the engine only where responsibilities are distinct: feed application, integrity checking and repair coordination. Preserve push-before-pull and the prohibition on mirror repair overwriting unsent work.

Desktop's SQLite outbox and phone's browser outbox are different persistence adapters. Their implementations should remain separate, with shared scenarios and operation contracts. Prefer parity tests over pretending one Python implementation can run directly on the phone.

Key verification: dropped acknowledgements, repeated operations, partial batches, deleted targets, parent rows arriving later, token changes, pending writes during repair and device-scoped timer recovery.

07 / Phone and native UI modules

Make the web entry point a coordinator

Move subjects, goals/templates, sessions and graphs into feature modules using the existing weight module as a starting pattern. Keep startup, navigation, shared refresh coordination and connectivity in `web/app.js`.

Define a small feature lifecycle: `mount(host, dependencies)`, `activate(context)`, `refresh(data)`, `deactivate()` and `destroy()`. Add only hooks that a real feature needs. A feature owns its event listeners, resize observers, draft state and request cancellation/epoch checks.

Inject API/outbox services and navigation actions; avoid feature modules importing the app coordinator. Shared state has a single owner. Separate canonical loaded data, queued projections, view preferences and unsaved drafts so a refresh cannot silently replace a form.

Extract reusable seams with clear scope

- Keep fullscreen behavior in the new `fullscreen.js` utility; plots own their resizing and controls.
- Keep agenda clock-position logic pure and testable; server context supplies logical-day settings.
- Put escaping, DOM helpers and display formatting in focused shared modules.
- Centralize browser storage keys and account-scoping helpers after verifying current behavior.
- Split CSS into tokens, shell, common controls and feature styles; consolidate selectors after visual regression checks.

Do not add a bundler or framework just to split ES modules. Maintain the service-worker shell inventory when asset paths change. Add a transitive module-import and CSS-asset coverage check so an online refactor cannot break offline launch. Measure startup/rendering cost; defer heavy graphs until opened.

Native UI

Keep `MainWindow` responsible for window lifecycle, dependency injection, navigation and sync coordination. The existing goals/subjects/graphs mixins can migrate into feature controllers or panels incrementally where that clarifies state ownership.

Retain shared session actions in `session_list.py`. Dialogs should report intent/results; services perform mutations. Pass a service/clock explicitly to new components rather than expanding access to a global window object.

Performance contract

Do not rebuild a whole feature for an unchanged context head. Pause hidden/unfocused rendering. Test fullscreen/rotation without destroying controls, stale network responses after navigation, and save responses arriving after a newer draft. Preserve the practical larger controls introduced in this phone update.

08 / Redundancy removal without behavior drift

Remove accidental duplication first

Candidate	Consolidation	Proof before removal
Repeated engineering instructions	One policy source with tool-specific pointers	All required build/install/data rules remain reachable.
Goal/subject/session form utilities	Shared parsing, labels and dialog helpers	Same validation, focus and confirmation behavior.
Date/display formatting	Focused utilities with explicit day semantics	Midnight and logical-day boundary fixtures.
Table/wire field descriptions	Generate from `sync_policy` contracts	Snapshot, import and feed mappings remain consistent.
Build/test command variants	One command dispatcher + thin wrappers	Uses a valid interpreter and protects active sessions.
Inline style tokens and repeated controls	Common tokens/control rules	Phone sizes, native colors and contrast remain correct.

Keep deliberate differences

Python and JavaScript weight math serve different runtimes. Preserve parity fixtures for LOESS and historic interpolation; do not replace the phone implementation with network-only calculation, which would break offline plots. A fixture generator can produce shared expected outputs from the pure Python implementation.

Native Qt and web controls also have different rendering systems. Share semantic choices and contract tests; do not force both through a CSS abstraction. The existing Qt color-alpha rules must remain explicit.

Weight_graph integration

Treat the sibling project as an external input/output adapter. `server/import_weights.py` owns format adaptation; `WeightService` owns persisted history rules; pure plot modules own mathematical behavior. Document the import/export bridge, source precedence and stable identity scheme.

Keep the original spreadsheets/CSV files immutable during imports. Do not make the repository's architectural tooling index their contents or copy personal measurements into reference fixtures.

Dead-code policy

A search with no references is evidence, not proof. Check dynamic imports, Qt signal wiring, entry points, service-worker assets and PyInstaller packaging before removing a module or asset. Delete confirmed unused code in small changes with relevant smoke tests and a successful packaged launch.

Rule: consolidate code when it owns the same rule or lifecycle. Similar-looking code with different contracts is not automatically redundant. Avoid a shared `utils.py` that becomes the next large mixed-responsibility file.

09 / Verification and development tooling

Make targeted checks discoverable

The feature manifest should map changes to focused checks: timers to time/recovery/sync tests; weight entry to daily aliases/picker/offline tests; graph UI to aggregation/agenda/fullscreen/browser checks. Focused checks accelerate iteration. They supplement the repository's required full suite and packaged verification.

Group tests by domain, contracts, server, sync and UI gradually. Move reusable clients/fixtures into `tests/support`; some tests currently import helpers from collected test modules. Preserve test names and check fixture scopes so reorganization does not introduce shared databases or hidden Qt dependencies.

The UI smoke module is a candidate for feature groups and shared fixtures. Measure where its runtime is spent before optimizing. Keep offscreen Qt tests for actual widget behavior rather than replacing them with string comparisons of the implementation.

Proposed tools

Tool	Purpose	Dependency budget
`scripts/dev.py`	Dispatch test/context/build/status commands	Python standard library; invoke existing tools.
`check_references.py`	Validate feature paths, symbols and test maps	Python AST + explicit JS-export support.
`check_boundaries.py`	Reject forbidden layer imports and new cycles	Standard-library AST; initial known exceptions recorded.
Contract fixture generators	Share server/phone expected scenarios	Existing Python + Node.
Documentation generator	Render tables/maps from manifests and registries	Standard library; deterministic text output.

Use a validated `--python` option or a configured interpreter for test/build commands. Avoid encoding one developer's absolute interpreter path in reusable public scripts. Optional GUI/build dependencies stay out of server runtime requirements.

Continuous checks

Begin with reference validity, import boundaries, Python tests and dependency-free Node tests. Add the browser smoke suite once its fixture server, browser executable and cleanup are reproducible. Run native packaging on macOS; this does not require adding Windows/Linux app support.

Generated artifacts should be deterministic and checked for drift. Add stronger lint/type checks incrementally to touched modules rather than blocking work on a repo-wide formatting rewrite. Use optional browser tests explicitly; never let a missing browser quietly masquerade as a successful browser run.

10 / Roadmap with reviewable stopping points

Estimates below are planning ranges in focused working days, not commitments. They assume one primary implementer, existing behavior preserved and verification included. Reassess after the first two phases; do not add ranges mechanically as a promised delivery date.

Phase	Deliverables	Estimate	Exit gate
A - Navigation	Thin AGENTS entry, docs index, glossary, feature manifest, reference checker, task context command	2-3 days	Each core feature has valid owner/test references; zero broken links/symbols in supported checks.
B - Boundaries	Dependency checker, write/time contracts, explicit operation map, small shared helpers, documented facade interfaces	2-4 days	No new forbidden imports; operation mappings covered; unchanged full-suite behavior.
C - Feature extraction	Web features, server routers/operations, low-state Python services, persistence seams, native shell reduction	6-10 days	One domain at a time; stable public paths and wire shapes; desktop/package/browser verification.
D - Consolidation	Timer extraction after prerequisite seams, test groups, dead-code removal, performance baselines, automated docs	3-5 days	No unsent-data loss; parity fixtures, metrics and release runbooks reproduced.

Dependencies and useful parallel work

Manifest and documentation work can run alongside pure helper extraction. Test characterization can run ahead of a module move. Keep file ownership explicit when multiple agents work so they do not both edit a large coordinator.

Services and persistence should not be split simultaneously for the same domain. Extract the service through the existing database facade first, then move its persistence behind a repository. This gives failures a narrower explanation and rollback boundary.

Timer extraction depends on explicit state ownership, stable transaction ownership and the write-contract map. It should be the last high-risk extraction, not the first.

Prioritize the first release

Ship Phase A and the boundary checks from B as an independently useful improvement. That already reduces search effort and document drift. Continue Phase C only while module splits demonstrate shorter task paths and clearer ownership.

Stop criterion: if a proposed abstraction adds more files, import hops or duplicated state without making a real task easier, simplify it before extending the pattern.

11 / Prioritized backlog: navigation and contracts

Each row is a proposed work item. Priority P0 means prerequisite/high-value, P1 means next, and P2 means later improvement. Effort is relative: S is roughly half a day, M about one day, L two or more days. Items overlap within the phase estimates.

ID	Priority / phase	Concrete item and acceptance result	Size
R01	P0 / A	Add root AGENTS with links to canonical policy, tests and release rules; remove copied instruction bodies.	S
R02	P0 / A	Add docs/index with start points for timer, goals, sessions, graphs, weight, sync and packaging.	S
R03	P0 / A	Create feature manifest covering current entry points, owners, operations, tables and targeted tests.	M
R04	P0 / A	Check paths/symbols/IDs and generated index drift; deliberately broken references fail the check.	M
R05	P1 / A	Add concise task context command; a feature lookup returns only its relevant files/contracts/checks.	S
R06	P0 / B	Document tracking logical days versus weight calendar days with tested boundary examples.	S
R07	P0 / B	Document atomic write/outbox/receipt/mirror ownership with lost-acknowledgement example.	S
R08	P0 / B	Check forbidden Python imports; preserve current no-Qt boundary in core/services and detect cycles.	M
R09	P0 / B	Build explicit operation mapping across server handlers, desktop queue methods and phone projections.	M
R10	P1 / B	Define compatible handler result categories; existing skip/refusal/failure JSON scenarios pass.	M
R11	P1 / B	Centralize selected storage-key/account-scoping helpers without losing pending edits across credential changes.	M
R12	P1 / B	Document facade signatures and state/transaction ownership before extracting implementations.	S
R13	P1 / B	Unify test/build/status command routing, preserving interpreter validation and active-session install guard.	M

Check the payoff early

Use representative tasks before and after: find the weight daily-save rule, identify the timer heartbeat operation, locate the agenda hour conversion, and find the backup restore tests. Record files inspected and time to identify the owner/test command. Compare equivalent tasks; a smaller number of files is useful only if the correct owner was found.

Do not claim a percentage reduction in AI tokens or development time before measuring it. The initial measurable gain should be valid direct references and fewer repeated broad searches.

12 / Prioritized backlog: implementation and cleanup

ID	Priority / phase	Concrete item and acceptance result	Size
R14	P0 / C	Extract web subjects/goals/sessions/graphs one at a time; app.js retains coordination and drafts survive refresh.	L
R15	P1 / C	Split shell/control/feature CSS; 320px phone and landscape controls remain usable without overflow.	M
R16	P0 / C	Split server read routers; authentication, route order, compression and final static mount remain correct.	M
R17	P0 / C	Extract domain operation handlers behind one registry; replay receipt and ordered batch tests unchanged.	L
R18	P0 / C	Extract subject/goal/recurrence services behind TrackerService; shared completion rules have one owner.	L
R19	P1 / C	Extract analytics queries and window resolution; aggregation/agenda/grouping fixtures remain identical.	M
R20	P0 / C	Extract schema/connection and entity persistence behind Database; nested atomic/outbox tests prove one commit owner.	L
R21	P1 / C	Reduce native MainWindow coordination by moving coherent feature wiring; shortcuts and sync lifecycle stay correct.	M
R22	P0 / D	Extract timer state/use cases after R07/R12/R20; crash/replay/switch/session-identity tests prove no lost session.	L
R23	P1 / D	Split oversized UI tests into feature groups and scoped fixtures in tests/support; no shared data or hidden dependency.	M
R24	P1 / D	Add shared contract fixtures for replay/refusal/daily-weight/time boundaries; Python and Node consume the same cases.	M
R25	P2 / D	Remove confirmed dead code/assets and duplicate helpers, with import/signal/SW/package checks before deletion.	M
R26	P1 / D	Automate reference/contract docs and SW asset coverage; publish reproducible performance reports without private data.	M

A sensible first implementation batch

R01-R05 plus R06-R09 make a useful navigation and protection release. Follow with a single low-state service or web feature extraction. Use that change to validate the proposed naming and lifecycle before replicating it.

Ownership rule

Every backlog item names a responsibility, not a person. Its implementing change must update the matching feature rows and tests. When delegating, assign disjoint files and make the coordinator integration the responsibility of one agent.

Maintain a small exception list for intentional cross-boundary adapters. Each exception needs a reason and a named contract; an unexplained allowlist entry is not a substitute for a clean boundary.

13 / Worked migration: timer reliability

Timer handling is sensitive because a UI action, durable local write, queued operation and later server replay may happen at different times. Preserve that sequence while changing file organization.

Current references to inspect

Start with `TrackerService`, `SyncedTrackerService`, `Database.atomic`, `server.ops.apply_op`, `SyncEngine`, `web/api.js` and `web/offline.js`. Use timer reliability, recovery and sync safety tests to characterize behavior before a move.

Reviewable sequence

1. Map start, stop, switch, heartbeat and recovery. Record which object owns the active session and which methods adopt persisted open rows.
2. Write down the captured timestamp/session-UID operation contract. Existing queued actions must never retarget a newer timer.
3. Introduce a timer service using the same injected database and clock semantics. Delegate from TrackerService; move one behavior at a time.
4. Keep local mutation and enqueue inside the existing atomic boundary. Force enqueue failure and confirm rollback preserves the prior timer.
5. Exercise lost response, app restart, overlapping open rows, server restart and late replay. Preserve unfinished rows and device-scoped recovery.
6. Run full checks, build, sign and launch on an isolated database; install only when the real app is idle and its outbox is safe.
7. Remove old implementation only after delegation parity is proven, then update feature references and generated documentation.

Rollback boundary

A structural extraction should be revertible without a database migration. Keep public imports, operation names and serialized fields stable. If a schema change becomes necessary, schedule it separately with an online backup and copied-database migration tests.

Do not roll back live data by replacing the database with an older copy after new writes. Revert code that still understands the unchanged schema, retaining committed history and unsent operations. Keep the previous signed app available as an archive for code rollback.

Risks to track

Duplicate active-state ownership, hidden global connections, broken Qt signal imports, altered receipt fingerprints, stale static assets, dropped browser drafts and dynamic imports missed by reference checks are the main structural risks. Each has a corresponding contract or smoke test; do not rely solely on a clean import graph.

14 / Acceptance scorecard and source index

Measure improvement without rewarding file churn

Measure	Baseline / scope	Initial target
Broken references	No feature/reference checker today	Zero broken supported paths, symbols and IDs.
Domain coverage in feature map	No canonical manifest today	All seven main areas: subjects, timer/sessions, goals, graphs, weight, sync, backups.
Forbidden core/service imports	Zero found in inspected AST	Preserve zero; automated check rejects new violations.
Broad coordinator size	app.js 1,577; TrackerService 1,148 audit lines	Reduce responsibilities, not an arbitrary line quota.
Task navigation effort	Not measured	Record four representative tasks before/after; owner and tests discoverable from one index.
Behavior preservation	Existing regression suites and fixtures	Full required suite plus applicable browser/package checks pass.
Unsent-data loss	Required invariant	Zero lost operations in crash/retry/repair fixtures.
Documentation drift	Stale facts found during audit	Paths/contracts verified; change counts generated or omitted.

Repository references

- `jobtracker/core/database.py`: Database, atomic transaction scope, schema setup and backup merge.
- `jobtracker/core/timeutils.py`: parsing, logical-day and duration helpers.
- `jobtracker/core/sync_policy.py`: wire/table identity and shared settings policy.
- `jobtracker/services/tracker_service.py`: domain facade and aggregation rules.
- `jobtracker/services/weight_service.py`: daily weight and identity alias behavior.
- `jobtracker/sync/{service,engine,state,client}.py`: local write adapter, synchronization, queue and transport.
- `server/{app,ops,feed,auth}.py`: API composition, operations, feed triggers and authentication.
- `web/{app,api,offline,weight,weight-utils,fullscreen}.js`: phone orchestration, queue, projection and features.
- `jobtracker/ui/{app,goals_mixin,subjects_mixin,graphs_mixin}.py`: native shell and existing feature seams.
- `jobtracker/ui/widgets/session_list.py`: common native session actions.
- `tests/test_operations_coverage.py`, `test_sync_safety.py`, `test_timer_reliability.py`, `test_weight_daily_edges.py`, `tests/mobile/`: meaningful compatibility checks.
- `deploy/install.sh`, `build_macos.sh`, `JobTracker.spec`: deploy/build/install paths that must stay coordinated.

The editable Markdown companion is the searchable version of this proposal; the PDF is its review copy. Neither replaces the proposed live manifest. Implement the reference system with source validation, then update it alongside code changes.